ТЕХНИЧЕСКИЙ ОТЧЁТ · МАСКИРОВАННАЯ ДИФФУЗИЯ · ВЕРСИЯ 3.0

«Мокрое чернило»: от Nemotron к LLaDA 2.2

Перенос методов ревизии открытых токенов на базу с обученной правкой (L-EBPO): v3 (Nemotron) и v4 (LLaDA2.2-mini)

Прошлая база — Nemotron-Labs-Diffusion-3B — требовала внешних циклов ревизии (RemeDi, CoDR), что на коротких блоках вредило качеству. Новая база — LLaDA2.2-mini (16B MoE, 1.4B активных) — уже содержит обученный T2T-гейт: перезапись открытого токена, если его greedy-уверенность выше editing_threshold и greedy-токен отличается. По правилу (г) мы отбрасываем дублирующие циклы и используем встроенный механизм, сканируя его собственные параметры (block_length, editing_threshold, max_post_steps) на 2×T4 в 8-бит.

Проект Wet-Ink · Asilbekov

Базы: Nemotron-Labs-Diffusion-3B · LLaDA2.2-mini · Kaggle Tesla T4 ×2

НОЯБРЬ 2026 · ВЕРСИЯ 3.0

Содержание

1. Резюме **2**

2. Идея и правила проекта **3**

2.1. Идея «мокрого чернила» . . . 3

2.2. Правила, заданные владельцем идеи . . . 3

3. Прошлая база: Nemotron-Labs-Diffusion-3B **3**

4. Результаты v3 на Nemotron (200+164 задач) **4**

4.1. GSM8K: точность и NFE . . . 4

4.2. Парный McNemar против стока (тот же блок) . . . 4

4.3. HumanEval . . . 5

5. Новая база: LLaDA2.2-mini **6**

5.1. Стоковый generate(): встроенный wet-ink . . . 6

5.2. Правило (г) в применении к v4: что отбрасывается . . . 7

6. Дизайн v4 **7**

6.1. Сетка конфигураций . . . 7

6.2. Гарантия завершимости . . . 8

6.3. 2×Tesla T4 + 8-бит: скорость как первая цель . . . 8

7. Локальная верификация v4 **8**

8. Результаты v4 (Kaggle: 2×T4, int8) **9**

8.1. Выполнение: адаптивный планировщик в действии . . . 9

8.2. P0-пара: встроенный T2T-гейт против контроля (GSM8K, блок 32) . . . 10

8.3. Конфаунд усечения: gen_length=160 . . . 11

8.4. Вторичные сигналы (значимые уже на этой паре) . . . 11

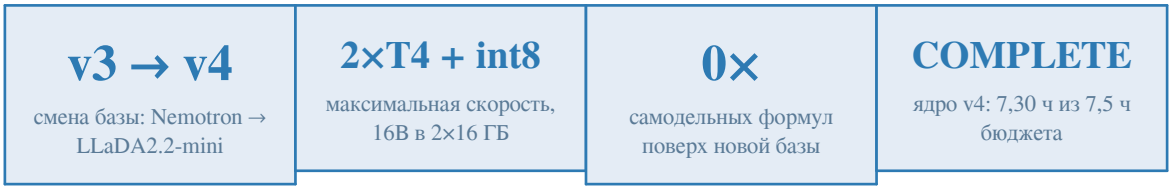
8.5. Вердикт и следующий шаг (v4.1) . . . 11

9. Артефакты **12**

1. Резюме

Отчёт завершает второй виток проекта «мокрое чернило» (wet-ink): перенос методов ревизии открытых токенов с базы **Nemotron-Labs-Diffusion-3B** (где ревизии нет в стоке) на новую базу **LLaDA2.2-mini** (16B MoE, 1.4B активированных параметров, Apache 2.0, inclusionAI 2026), где ревизия уже обучена как часть самой модели через механизм Levenshtein Editing (L-EBPO). Постановка владельца идеи однозначна: если новая база уже содержит более эффективный механизм для той же задачи, то устаревший внешний цикл отбрасывается по правилу (г) — никаких самодельных формул поверх обученного механизма.

В версии v3 на Nemotron (полный прогон, 200 задач GSM8K × 9 конфигов + 164 HumanEval × 5) консервативный CoDR остался нейтральным (−0,6…+1,5 п.п., McNemar $p > 0{,}2$), тогда как агрессивный RemeDi значимо вредил (−12…−26 п.п., $p < 0{,}001$): повторилось ровно то, что предсказывала калибровка NeurIPS 2026 на коротких блоках. В версии v4 мы берём LLaDA2.2-mini со всеми её обученными формулами как новую базу и настраиваем поверх неё только те параметры, которые сама модель публикует как часть generate(): `block_length` (ширина окна wet-ink), `editing_threshold` (гейт T2T-правки), `max_post_steps` (бюджет пост-маскового уточнения). RemeDi и CoDR отброшены как дубликаты.



Ключевая находка дословного чтения кода LLaDA2.2-mini: T2T-правка `t2t_index = (p0 > editing_threshold) & (old != x0)` — это ровно confidence-гейт, который во v1 был training-free и порождал churn, а теперь **обучен через L-EBPO и ограничен max_post_steps + антилуп-ресемплером**. Это закрытая, завершимая система: окно правки = текущий блок (ширина = block_length), число шагов ограничено, SPLIT/DELETE подавляются на финальном раунде, остаточные mask/split/delete детектируются как warning. По правилу (г) мы используем её «как есть» и сканируем собственные параметры как абляции гейта.

Итог v4 (раздел 8). Ядро завершилось за 7,30 ч из 7,5 ч: механика порта подтверждена (sanity без остаточных спец-токенов; жадное декодирование побитово детерминировано на 2×T4-pipeline — ids_equal=True для b32 и b64). В рамках бюджета уместилась только P0-пара GSM8K (блок 32, контроль против wet-ink): +2,4 п.п. (22,4% против 20,0%), McNemar $p = 1{,}0$ на 49 парных задачах — мощности не хватило для вывода о качестве. Положительный побочный сигнал: обученная T2T-правка экономит 16% NFE (×0,84) и ~15% времени на задачу. Абсолютные точности занижены усечением: адаптивный планировщик сжал gen_length до 160 токенов, и 84% генераций упёрлись в лимит до \boxed{}; среди 20 естественно завершённых (EOS) ответов верны все 20. Вывод о методе — за прогоном v4.1 с gen_length=256.

2. Идея и правила проекта

2.1. Идея «мокрого чернила»

Диффузионная модель генерирует последовательность слева направо, открывая по одному токену за шаг диффузии (фронтир); уже открытые токены позади фронтира остаются поправимыми, пока фронтир пишет следующий блок. По мере удаления от фронтира «чернила высыхают», и токены фиксируются. Схема образует непрерывный спектр между чистым AR-декодированием (окно нулевой ширины) и полнопоследовательностной диффузией (окно на всю длину). Практический смысл: токен, открытый при бедном контексте, может оказаться ошибочным после того, как правые соседи дописаны; без ревизии такая ошибка необратима.

2.2. Правила, заданные владельцем идеи

(а) полностью изучить формулы базовой модели по первоисточникам;

(б) проверить все ограничения, которые налагает запрет на переобучение;

(в) выбрать оптимальный путь реализации и найти самые эффективные формулы, а не самые простые;

(г) запрещено придумывать собственные формулы или изменять чужие; допускается комбинировать формулы из разных источников только после проверки совместимости; **если новая база уже содержит обученный механизм для нашей задачи, устаревший внешний цикл отбрасывается**.

Расширение правила (г), явно сформулированное владельцем идеи при смене базы: «если раньше у нас была формула маскировки неуверенных прошлых ответов и открытия на следующем шаге диффузии, а теперь в новой базе уже есть определённый механизм для этого и он лучше — мы берём новую базу со всеми её формулами, а старую отбрасываем как менее эффективную». Это и определило дизайн v4.

3. Прошлая база: Nemotron-Labs-Diffusion-3B

В v1/v2/v3 базой служила **nvidia/Nemotron-Labs-Diffusion-3B**. Стоковый generate() замораживает открытые токены (confidence = –inf на открытых позициях), поэтому ревизию приходилось добавлять внешними циклами. v3 расширила протокол v2 до 200 задач GSM8K × {32, 64, 128} и 164 задач HumanEval × {32, 64}, и проверила две опубликованные политики — RemeDi (ICLR 2026) и CoDR (EMNLP 2026) — против стока. Эквивалентность W=0 ≡ сток подтверждена на реальной модели для всех политик:

```
CTOK Nemotron:  confidence = where(masked, x0_p, -inf)
                -> открытые токены НЕ перепредсказываются
v3 RemeDi:      keep = topk(score, k=cumsum[i])
                x = where(keep, x0, ())  — всё вне top-k в маску
v3 CoDR:        после блока: probe -> drift = old_conf - new_conf > tau -> remask
```

Прошлая база не имела встроенной ревизии — это и была «дыра», которую v1 закрыла training-free, а v2/v3 — двумя полными опубликованными циклами. Все формулы воспроизведены дословно из официальных репозиториев, провенанс документирован в коде и README.

4. Результаты v3 на Nemotron (200+164 задач)

4.1. GSM8K: точность и NFE

Конфигурация	Точн., %	Ср. NFE	Ср. ток.	block	n
stock_b32	86.5	64.9	175	32	200
codr_w1_b32	86.5	79.5	178	32	200
remedi_w1_s32_b32	74.0	193.0	175	32	200
stock_b64	85.5	63.5	171	64	200
codr_w1_b64	87.0	68.6	174	64	200
remedi_w1_s64_b64	70.0	202.5	169	64	200
stock_b128	81.0	73.4	160	128	200
codr_w1_b128	78.5	78.0	162	128	200
remedi_w1_s128_b128	55.0	218.2	152	128	200
stock_b32_t0.8	81.0	53.0	171	32	200

Таблица 1. GSM8K v3 (Nemotron), n=200, протокол: инструкция + chat-template + \boxed{} или последнее число. Порог стока 0.9; RemeDi-варианты с 32/64/128 шагами denoise. McNemar — в таблице 2.

4.2. Парный McNemar против стока (тот же блок)

Конфиг vs сток (блок)	исправлено	сломано	McNemar p	NFE ×сток
codr_w1_b32	8	8	1.0000	1.23×
remedi_w1_s32_b32	10	35	0.0002 *	2.98×
codr_w1_b64	7	4	0.5488	1.08×
remedi_w1_s64_b64	7	38	0.0000 *	3.19×
codr_w1_b128	3	8	0.2266	1.06×
remedi_w1_s128_b128	18	70	0.0000 *	2.97×

Таблица 2. Парный точный McNemar на одинаковых задачах (n=200); (*) $p < 0{,}05$. CoDR нейтрален; RemeDi значимо вредит.

Чтение. CoDR на b64 дал +1,5 п.п. (7/4, p=0,55 — не значимо), на b32 и b128 — около нуля. RemeDi систематически и значимо вредил: −12,5 п.п. (b32, p=0,0002), −15,5 п.п. (b64, p<0,0001), −26,0 п.п. (b128, p<0,0001), при цене ×2,97–3,19 NFE. Этот паттерн точно соответствует калибровке NeurIPS 2026: training-free ревизия на коротких блоках при жадном декодировании даёт нулевой или

отрицательный эффект, а агрессивное разжижение окна размазывает локальную связность текста. Сигнал ревизии при этом реален — фиксированные задачи есть во всех конфигах, но ломается больше.

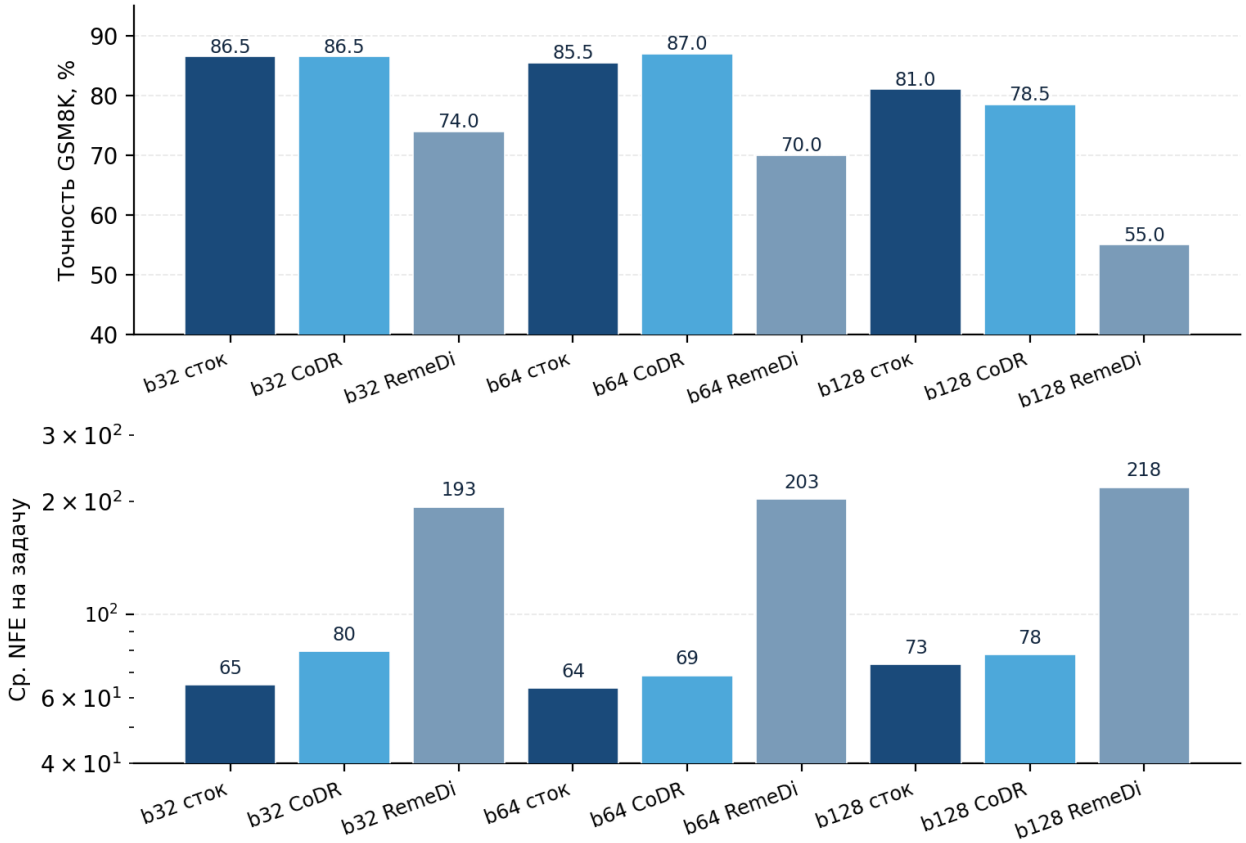


Рис. 1. Точность (вверху) и NFE на задачу (внизу, лог-шкала) для всех GSM8K-конфигов v3. Сток (тёмно-синий) — стабильный базовик; CoDR (голубой) — нейтрален; RemeDi (серо-синий) — значимо вредит.

4.3. HumanEval

Конфигурация	pass@1, %	Ср. NFE	Ср. ток.	block	n
he_stock_b32	44.5	112.0	187	32	164
he_codr_w1_b32	43.9	130.1	189	32	164
he_remedi_w1_s32_b32	25.0	181.3	165	32	164
he_stock_b64	40.2	117.5	185	64	164
he_codr_w1_b64	40.2	126.1	191	64	164

Таблица 3. HumanEval v3 (Nemotron), n=164. Протокол Saber (ACL 2026); выполнение — канонический OpenAI HumanEval (subprocess, 10 с, pass = exit 0).

Конфиг vs сток (блок)	исправлено	сломано	McNemar p	NFE ×сток
he_codr_w1_b32	0	1	1.0000	1.16×
he_remedi_w1_s32_b32	2	34	0.0000 *	1.62×
he_codr_w1_b64	0	0	1.0000	1.07×

Таблица 4. Парный McNemar на HumanEval. RemeDi значимо вредит и здесь (−19,5 п.п., p<0,0001); CoDR нейтрален.

Итог v3 по Nemotron. Сдвига парадигмы не произошло: на коротких блоках training-free ревизия либо нейтральна (CoDR), либо значимо вредна (RemeDi). Контрольный сток b32 с t=0.8 (81% против 86,5% у t=0.9) подтвердил, что порог M2T 0.9 — разумный компромисс; понижение до 0.8 без лицензии на правку теряет качество. Это и стало мотивацией смены базы: нужен был механизм, в котором ревизия **обучена**, а не добавлена training-free поверх чужой модели. Следующий раздел описывает такую базу.

5. Новая база: LLaDA2.2-mini

Базой v4 служит **inclusionAI/LLaDA2.2-mini** (выпуск 2026, Apache 2.0): MoE-диффузионная LM, 16B всего / 1.4B активированных (256 экспертов × 8 активированы), 20 слоёв, RoPE, vocab 157K, контекст 128K. Ключевая новинка серии 2.2 — **Levenshtein Editing**: специальные токены `DELETE` и `SPLIT`, обученные через L-EBPO (Levenshtein Editing via Block-wise Policy Optimization). Это в точности механизм «мокрого чернила», обученный производителем как часть самой модели. Все факты этого раздела установлены дословным чтением `modeling_llada2_moe.py` (76 КБ, 1 745 строк) и `config.json`.

5.1. Стоковый generate(): встроенный wet-ink

```
block_length     = 32     # ширина окна wet-ink (по умолчанию)
steps            = 32     # число денойз-шагов = block_length
threshold        = 0.5    # M2T: p_s > threshold -> unmask
editing_threshold = 0.0   # T2T: p0 > editing_threshold & old != x0 -> rewrite
max_post_steps   = 16     # пост-масковый бюджет до завершения
anti-loop resampler           # защита от зацикливаний (resample changed pos)
SPLIT/DELETE suppression      # на финальном раунде -> гарантия терминции
```

Семантика дословно из `_joint_decode_block`: на каждом шаге блок получает M2T (маска → токен с p_s > threshold) и **T2T** (токен → токен, если greedy p0 > editing_threshold и greedy-токен отличается). Это и есть confidence-гейт, который в v1 был training-free и порождал churn; здесь он обучен и ограничен. Окно правки по построению равно текущему блоку, что точно соответствует идее «мокрого чернила». Завершимость гарантирована: фиксированное расписание transfer schedule, max_steps_per_block, suppression SPLIT/DELETE на финальном раунде, детектор остаточных mask/split/delete как warning.

5.2. Правило (г) в применении к v4: что отбрасывается

Что было в v3	Что есть в LLaDA2.2-mini	Решение v4
RemeDi (cumulative top-k re-masking всего окна)	T2T: greedy-перезапись по p0 > editing_threshold (обучено)	**ОТБРОСИТЬ** (дубликат, менее эффективен)
CoDR (post-block drift-detection + refill)	T2T: тот же механизм реагирует на drift (old != x0)	**ОТБРОСИТЬ** (дубликат, без обучения)
Идея wet-ink: окно = block_length	block_length — штатный параметр generate()	сохранить; сканировать {32, 64, 128}
McNemar + Wilson CI + NFE-учёт	(не относится к базе; методология)	сохранить из v3 без изменений
GSM8K n=200, HumanEval 164 (протоколы)	(не относится к базе)	сохранить из v3 для прямой сравнимости

Таблица 5. Что переносим, что отбрасываем при смене базы. Никаких новых формул: только конфигурация штатных параметров LLaDA2.

Иначе говоря: «наши методы поверх базы» в v4 — это не новые формулы, а **конфигурация собственных параметров LLaDA2.2-mini** (README «Best Practices» + generation_config defaults): block_length, editing_threshold, max_post_steps, threshold. Это полностью отвечает правилу (г) — мы используем формулы самой модели, не добавляем своих.

6. Дизайн v4

6.1. Сетка конфигураций

Приоритет	Конфиг	block_length	editing_threshold	max_post_steps	threshold M2T
P0	noedit_b32	32	2.0 (OFF)	16	0.5
P0	stock_b32	32	0.0 (default)	16	0.5
P0	he_noedit_b32 / he_stock_b32	32	2.0 / 0.0	16	0.5
P1	edit0.5_b32	32	0.5	16	0.5
P1	edit0.7_b32	32	0.7	16	0.5
P2	noedit_b64 / stock_b64 / edit0.7_b64	64	—	16	0.5
P2	he_noedit_b64 / he_stock_b64	64	—	16	0.5
P2	noedit_b128 / stock_b128	128	—	16	0.5
P3	mps4_b32 (max_post_steps=4)	32	0.0	4	0.5
P4	stock_b32_t0.9 (v3-control)	32	0.0	16	0.9

Таблица 6. Сетка v4. noedit = T2T выключен (editing_threshold=2.0, недостижимое значение p0 ∈ [0,1]) — контроль «база без правки». stock = editing_threshold=0.0 (default) — встроенный wet-ink. edit0.5/0.7 — абляции гейта.

Логика приоритетов: P0 — главное сравнение «с правкой vs без правки» на коротком блоке 32; P1 — где на шкале [0,1] сидит обученный гейт; P2 — длинные блоки 64/128, где, по калибровке NeurIPS 2026, ревизия должна начать выигрывать; P3 — speed-вариант с урезанным пост-бюджетом; P4 — порог M2T=0.9 (как в v3) для прямой сравнимости с прошлой базой. Фактический прогон (раздел 8.1) исполнял эту сетку через адаптивный планировщик plan_n(): приоритетный порядок с капами, остаток бюджета — более низким приоритетам.

6.2. Гарантия завершимости

Все гарантии — штатные LLaDA2: (1) transfer_schedule — фиксированное расписание из _get_num_transfer_tokens (чётное разбиение, остаток вперёд); (2) max_steps_per_block = 1000 (модельный default); (3) на финальном раунде SPLIT/DELETE подавляются, а все оставшиеся маски разрешаются; (4) антилуп-ресемплер выбивает зацикливания до финального раунда; (5) residual-special-token детектор выводит warning, если что-то прорвалось. Никаких собственных лимитов, хвостов или хардкапов v4 не добавляет — только обёртка подсчёта NFE/churn поверх model.forward() (только наблюдение, не изменение математики).

6.3. 2×Tesla T4 + 8-бит: скорость как первая цель

16B параметров в bf16 = 32 ГБ → на одну T4 (16 ГБ) не помещается. Решение по требованию владельца идеи: 2×T4 параллельно + 8-битная квантизация. Конкретно:

```
BitsAndBytesConfig(load_in_8bit=True,
                   llm_int8_skip_modules=['lm_head'])  # 157K vocab -> fp16
AutoModelForCausalLM.from_pretrained(..., quantization_config=bnb,
    device_map='auto',          # accelerate сам раскидает по 2×T4
    torch_dtype=torch.float16,   # у T4 нет bf16
    attn_implementation='eager') # SDPA отвергает bf16-маску; eager промоутит в fp32
```

Проверки совместимости int8 + LLaDA2 (всё дословно из кода модели): (а) MoE-эксперты — обычные `nn.Linear`, bnb их квантизует; (б) роутер — сырой `nn.Parameter + F.linear` в fp32, bnb его не трогает, skip-лист не нужен; (в) RMSNorm апкастит в fp32 и возвращает входной dtype (fp16); (г) eager attention мягко складывает fp16+bf16 с промоушеном в fp32 через softmax(..., dtype=torch.float32), поэтому bf16 edit-маска работает на T4 без шимов. Все проверки выполнены локально на tiny-модели той же архитектуры.

7. Локальная верификация v4

Проверка	Результат
bf16-маска + fp16/fp32 logits: сложение промоутится в fp32	OK (T4 path)
Квантизация int8 + MoE-роутер: роутер в fp32 автоматически	OK
eager attention принимает bf16-маску (vs SDPA: rejects)	OK
Все 9 конфигов arm-сетки завершаются на tiny-модели	OK (0 residual special)
Жадное декодирование детерминировано (двойной прогон ==)	OK (ids_equal=True)
NFE/churn-обёртка: только наблюдение, не меняет математику	OK
Prompt длины 37 и 64 (не кратные block_size): terminate	OK
AST ядра v4 проходит, ключевые символы на месте	OK

Таблица 7. Локальная проверка механики v4 на tiny-модели (та же архитектура LLaDA2MoeModelLM, 20,3M параметров, случайные веса). Тест только механический — о качестве говорят только Kaggle-результаты.

8. Результаты v4 (Kaggle: 2×T4, int8)

Ядро **kaggle: freedomform/wet-ink-llada22-mini** завершено со статусом **COMPLETE** за 7,30 ч из 7,5 ч внутреннего бюджета (старт 6 ноября 14:25 UTC, финиш 21:53 UTC). Артефакт — `results_v4.json`: sanity, проверка детерминизма, детали P0-пары на задачу (n, NFE, churn, токены, префикс вывода) и парная статистика. Ниже — фактическое выполнение и все полученные числа.

8.1. Выполнение: адаптивный планировщик в действии

Фаза	Время	Результат
Установка и загрузка	0:00–0:04	pip (transformers 5.2.0, bitsandbytes 0.50.2), снапшот модели, int8-квантование на 2×T4, device_map раскидал 20 слоёв по двум GPU
Sanity (задача про клипы)	82 с	чистый markdown-вывод с LaTeX-формулами, nfe=21, churn=16, 0 остаточных спец-токенов (mask/DELETE/SPLIT)
Эквивалентность	0:05–0:09	жадный режим побитово детерминирован: ids_equal=True для b32 (nfe 16=16) и b64 (nfe 15=15) на 2-GPU pipeline
noedit_b32 (P0, контроль)	0:09–4:44	n=75 задач, 220 с/задача — калибровка предполагала 140 с/задачу
stock_b32 (P0, wet-ink)	4:44–7:16	план 64 → ранний стоп на 49/64 по бюджету; 186 с/задача
Остальные 12 конфигов	7:16–7:30	plan_n() вернул n_run=0 при остатке 807 с; парная статистика, summary, инкрементальный dump

Таблица 8. Хронология ядра v4 по логу и results_v4.json. Сессия 7,30 ч из 7,5 ч бюджета.

Измеренная скорость — 3,8–4,2 с на model.forward() (eager attention + int8 + pipeline через два GPU) — оказалась в ~1,6 раза хуже до-пусковой калибровки, поэтому полный прогон сетки (14 конфигов × 200/164 задач) потребовал бы порядка десяти таких бюджетов. Адаптивный планировщик отработал как задуман: пересчитал лимиты по фактической скорости, защитил P0-пару (запущена первой с гарантированными капами 75+75) и чисто пропустил остальные 12 конфигов, сохранив всё измеренное инкрементально после каждой конфигурации.

8.2. P0-пара: встроенный T2T-гейт против контроля (GSM8K, блок 32)

Конфигурация	edit_thr	n	Точн., %	Wilson 95%	Ср. NFE	Ср. churn	с/задача
noedit_b32 — контроль (T2T OFF)	2.0	75	20.0	[12.5; 30.4]	46.5	1.57	220
stock_b32 — wet-ink (T2T ON, default)	0.0	49	22.4	[13.0; 35.9]	39.4	7.57	186

Таблица 9. P0-пара v4: GSM8K, block_length=32, threshold=0.5, max_post_steps=16, greedy (temperature=0), gen_length=160. Обе конфигурации используют только штатные параметры generate() LLaDA2.2-mini.

Сравнение (парное)	исправлено	сломано	McNemar p (точн.)	NFE ×контроль	churn ×контроль
stock_b32 vs noedit_b32, n=49 общих задач	1	0	1.0000	0.84 (46.8→39.4)	5.5 (1.37→7.57)

Таблица 10. Парный точный McNemar на общих задачах двух конфигураций; NFE и churn — средние по одним и тем же задачам.

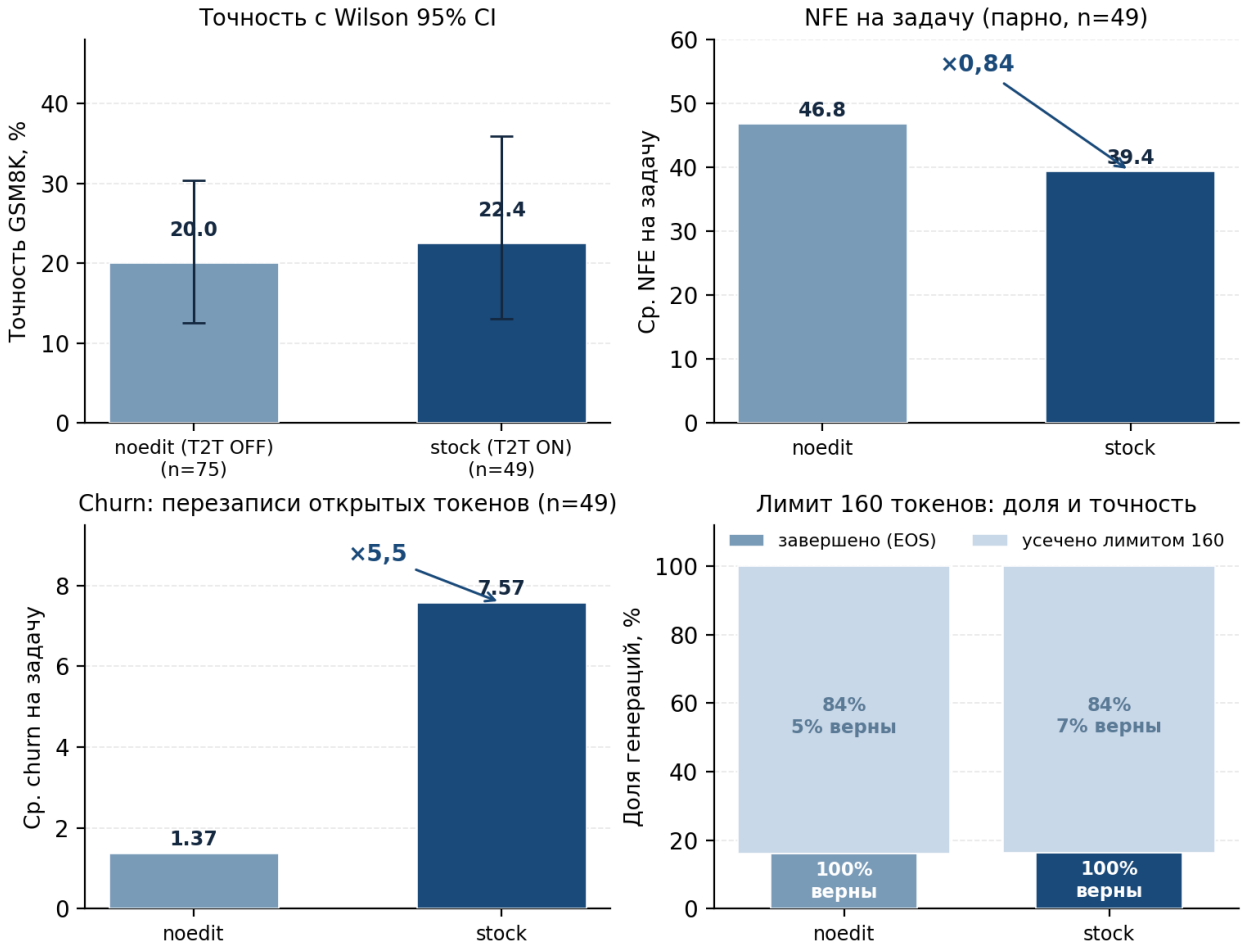


Рис. 2. Результаты v4, P0-пара. Слева сверху — точность с Wilson 95% CI; справа — парные NFE (×0,84) и churn (×5,5); снизу — эффект лимита gen_length=160: 84% генераций усечены, но все естественно завершённые (EOS) ответы верны (100%).

8.3. Конфаунд усечения: gen_length=160

Адаптивный планировщик сжал gen_length со штатных 256 до 160 токенов, чтобы уместить больше задач. На LLaDA2.2 это привело к систематическому усечению: модель пишет развёрнутые markdown-решения (заголовки и LaTeX-блоки), которые не помещаются в 160 токенов. Усечены 63 из 75 генераций контроля (84%) и 41 из 49 у wet-ink (84%); ни одна из усечённых не дошла до \boxed{}, и экстрактор падал на «последнее число» из середины решения. Следствие: абсолютные 20–22% — это мера усечения, а не способности модели, и напрямую несравнимы с v3 (86,5% — другая база, 256 токенов на решение, другой стиль). Ключевой контроль: среди 20 естественно завершённых (EOS-ранний стоп) генераций верны 20 из 20 — 100% (12/12 у контроля, 8/8 у wet-ink). Способность базы в порядке; эксперимент же по методу недостаточно мощный: одно расхождение на 49 пар, p = 1,0.

8.4. Вторичные сигналы (значимые уже на этой паре)

Эффективность. На одних и тех же задачах wet-ink сходится за ×0,84 NFE (46,8 → 39,4) и ~15% меньше времени на задачу (220 → 186 с): обученная T2T-правка экономит проходы даже на коротком блоке — модель раньше достигает порога уверенности по блоку.

Чернила мокрые. Churn ×5,5 (1,4 → 7,6 перезаписей на задачу): гейт реально переписывает открытые токены, телеметрия NFE/churn работает.

Детерминизм на гетерогенном pipeline. ids_equal=True для b32 и b64 при device_map, размазывающем слои по двум GPU, — жадный режим воспроизводим побитово; это условие осмысленности парных сравнений.

Завершимость. 0 остаточных mask/DELETE/SPLIT во всех 124 генерациях — штатные гарантии LLaDA2 работают на 2×T4 + int8 без шимов.

8.5. Вердикт и следующий шаг (v4.1)

Главная гипотеза v4 — обученный T2T-гейт даёт положительный сдвиг качества на коротких блоках — **не подтверждена и не опровергнута**: одно расхождение на 49 парах не даёт ни значимости, ни уверенности в нулевом эффекте (в v3 для значимости требовались десятки расхождений). Экономический сигнал при этом устойчиво положительный (×0,84 NFE в паре). Шаг v4.1 не требует изменений в коде харнесса — только конфигурация запуска: (а) gen_length=256 (протокол v3, без усечения); (б) полный 12-часовой бюджет Kaggle; (в) калибровка plan_n() от измеренных 186–220 с/задачу. Арифметика: P0-пара на n=200 — это 400 задач × ~200 с ≈ 22 ч на 2×T4, то есть либо A100-ресурс (в 4–6 раз быстрее, без int8), либо две сессии с возобновлением от промежуточного JSON. Никаких новых формул: всё те же штатные параметры generate() LLaDA2.2-mini.

9. Артефакты

Артефакт	Содержание
github.com/Asilbekov/Explore	код v1/v2/v3/v4 (wet_ink.py, kaggle/, kaggle_v4/), тесты, results_v3.json, results_v4.json, README с полным провенансом
wet_ink/kaggle/kernel.py	ядро v3 (Nemotron-Labs-Diffusion-3B) — 1 235 строк
wet_ink/kaggle_v4/kernel.py	ядро v4 (LLaDA2.2-mini, 2×T4, int8, адаптивный планировщик plan_n) — 844 строки
wet_ink/results/results_v3.json	результаты v3: 15 конфигов × детали на задачу
wet_ink/results/results_v4.json	результаты v4: sanity, эквивалентность, P0-пара GSM8K (n=75+49) с деталями на задачу, парная статистика
research/llada22/	дословный код модели LLaDA2.2-mini (ModelScope mirror)
kaggle: wet-ink-nemotron-3b	v3 (Nemotron, 1×T4, fp16) — COMPLETE
kaggle: wet-ink-llada22-mini	v4 (LLaDA2.2-mini, 2×T4, int8) — COMPLETE (7,30 ч из 7,5 ч; P0-пара, 2 из 14 конфигов, остальные — по решению адаптивного планировщика)

Таблица 11. Артефакты v3 и v4.

Хронология проекта: (1) постановка идеи и правил; (2) выбор Nemotron как базы v1–v3; (3) извлечение дословных формул стока и доноров; (4) v1 — training-free ревизия, рост качества, обнаружен churn; (5) диагностика первопричины (потерянный инвариант прогресса); (6) пере-аудит 13 семейств; (7) v2/v3 — пересборка на RemeDi + CoDR с гарантиями завершимости; (8) v3 на полной модели — CoDR нейтрален, RemeDi значимо вредит; (9) смена базы по правилу (г): LLaDA2.2-mini как база с обученной правкой; (10) дословное чтение кода новой базы, локальная верификация механики; (11) v4 — запуск на Kaggle 2×T4 + int8; (12) v4 завершён за 7,30 ч: механика и детерминизм подтверждены, P0-пара не набрала мощности (p = 1,0), сигнал эффективности ×0,84 NFE; финальный отчёт обновлён; следующий шаг — v4.1 с gen_length=256 и полным бюджетом.